BITS Pilani

Pilani | Dubai | Goa | Hyderabad

ASSIGNMENT SUBMISSION

AIMLC ZC416 - MFML

Assignment - 1

Submitted By	–	**Ghetiya Niraj Rajeshbhai**
Enrollment Number	–	**2025AC05976**

"ज्ञानं परमं बलम्"

Knowledge is Supreme Power

Assignment - 1

Date...../...../.....

Page.....

AIMLC ZC416 - MFML

1 - Finding solution of linear systems

(a) Write a code taking as input a matrix A of size mxn and b of size mx1, where m and n are arbitrarily large number and m<n constructing the augmented matrix and performing

(i) RGF

code:-

```
def ref(M):
    M = [row[:] for row in M]
    rows, cols = len(M), len(M[0])
    pivot_row = 0

    for col in range(cols):
        found = -1
        for r in range(pivot_row, rows):
            if abs(M[r][col]) > 1e-10:
                found = r
                break
        if found == -1:
            continue
        M[pivot_row], M[found] = M[found], M[pivot_row]

        for r in range(pivot_row+1, rows):
            factor = M[r][col] / M[pivot_row][col]
            for c in range(cols):
                M[r][c] = M[r][c] - factor * M[pivot_row][c]
        pivot_row += 1

    return M
```

Date...../...../.....

Page.....

(ii) RREF

code :-

```
def rref(M):
    M = ref(M)
    rows, cols = len(M), len(M[0])

    for pivot_row in range(rows-1, -1, -1):
        pivot_col = -1
        for c in range(cols)
            if absolute(M[pivot_row][c]) > 1e-10:
                pivot_col = c
                break
        if pivot_col == -1:
            continue

        piv = M[pivot_row][pivot_col]
        for c in range(cols):
            M[pivot_row][c] = M[pivot_row][c] / piv

        for r in range(pivot_row):
            factor = M[r][pivot_col]
            for c in range(cols):
                M[r][c] = M[r][c] - factor * M[pivot_row][c]

    return M
```

Date...../...../.....

Page.....

b Identify the pivot and non pivot columns and find the particular solution and solutions to $Ax=0$

code:-

```
def find_pivots(R, n_cols_A):
    pivot_cols, pivot_rows = [], []
    for r in range(len(R)):
        for c in range(n_cols_A):
            if abs(R[r][c]) > 1e-10:
                pivot_cols.append(c)
                pivot_rows.append(r)
                break
    non_pivot_cols = [c for c in range(n_cols_A) if c not in pivot_cols]
    return pivot_cols, non_pivot_cols, pivot_rows

def particular_solution(R_rref, pivot_cols, pivot_rows, n):
    x = [0.0]*n
    for i, pc in enumerate(pivot_cols):
        x[pc] = R_rref[pivot_rows[i]][-1]
    return x

def null_space_solutions(R_ref, pivot_cols, non_pivot_cols,
                         pivot_rows, n):
    solutions = []
    for fc in non_pivot_cols:
        x = [0.0]*n
        x[fc] = 1.0
        for i, pc in enumerate(pivot_cols):
            x[pc] = -R_rref[pivot_rows[i]][fc]
        solutions.append(x)
    return solutions
```

Date...../...../.....

Page.....

(c) Consider a random 6x9 matrix A and a suitable b and show the REF, RREF, pivot columns, non-pivot columns, the particular solution, the solutions to Ax=0 the general solution and verify the general solution.

$A =$

	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9
r_1	1	-2	5	2	-1	1	4	-3	1
r_2	5	5	2	-1	-2	2	2	-3	0
r_3	-1	-4	2	0	-4	-1	-5	4	0
r_4	3	-5	5	5	4	-3	1	-2	3
r_5	-3	-1	-3	1	-1	3	1	-4	-2
r_6	3	-4	4	3	4	-1	-4	-2	1

$b = [19, 24, -15, 15, -18, 1]$

==REF==

[A|b] REF =

$$\left[\begin{array}{ccccccccc|c} 1 & -2 & 5 & 2 & -1 & 1 & 4 & -3 & 1 & 19 \\ 0 & 15 & -23 & -11 & 3 & -3 & -18 & 12 & -5 & -71 \\ 0 & 0 & -2 & -2 & -4 & -1 & -8 & 5 & -1 & -24 \\ 0 & 0 & 0 & 8 & 21 & -1 & 21 & -16 & 4 & 56 \\ 0 & 0 & 0 & 0 & -5 & 3 & -1 & -3 & 2 & -11 \\ 0 & 0 & 0 & 0 & 0 & 3 & -2 & -4 & 2 & -9 \end{array}\right]$$

==RREF==

[A|b] RREF =

$$\left[\begin{array}{ccccccccc|c} 1 & 0 & 0 & 0 & 0 & 0 & -2 & 0 & 0 & -3 \\ 0 & 1 & 0 & 0 & 0 & 0 & 2 & 0 & 0 & 8 \\ 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 6 \\ 0 & 0 & 0 & 1 & 0 & 0 & 2 & -1 & 0 & 5 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & -1 & 0 & -2 \end{array}\right]$$

Date...../...../.....

Page.....

pivot_colums (1-indexed): [1, 2, 3, 4, 5, 6]

non-pivot colums (1-indexed): [7, 8, 9]

Rank(A) = 6, Nullity = 3

x_particular = [-3, 8, 6, 5, 0, -2, 0, 0, 0]

A.x_particular = b (True)

null_vec_1 = [2, -2, -1, -2, 0, 0, 1, 0, 0]

null_vec_2 = [0, 0, 0, 1, 0, 1, 0, 1, 0]

null_vec_3 = [0, 0, 0, 0, 0, 0, 0, 1]

A . null_vec_1 = 0 ✓

A . null_vec_2 = 0 ✓

A . null_vec_3 = 0 ✓

Date...../...../.....

Page.....

Q-2 matrix decompositions

(a) Consider a symmetric and positive definite matrix A of size nxn write code to construct an elementary matrix for every elementary row operation that is performed on A and using the theory explained in the class write the decomposition of A as LU, where L is a lower triangular matrix and U is an upper triangular matrix

matrix A =

34	40	24	21
40	68	40	36
24	40	34	26
21	36	26	27

code: LU Decomposition via Elemetary matrics

```
def LU_decomposition(A):
    n = len(A)
    U = [row[:] for row in A]
    L = [[1.0 if i==j else 0.0
          for j in range(n)] for i in range(n)]
    for col in range(n):
        for row in range(col+1, n):
            factor = U[row][col] / U[col][col]
            L[row][col] = factor
            for c in range(n):
                U[row][c] = U[row][c] - factor * U[col][c]

    return L, U

L, U = LU_decomposition(A)
```

Date...../...../.....

Page.....

$$L = \begin{bmatrix} 1.00 & 0.00 & 0.00 & 0.00 \\ 1.17 & 1.00 & 0.00 & 0.00 \\ 0.70 & 0.56 & 1.00 & 0.00 \\ 0.61 & 0.53 & 0.46 & 1.00 \end{bmatrix}$$

$$U = \begin{bmatrix} 34.00 & 40.00 & 24.00 & 21.00 \\ 0.00 & 20.94 & 11.76 & 11.29 \\ 0.00 & 0.00 & 10.44 & 4.83 \\ 0.00 & 0.00 & 0.00 & 5.70 \end{bmatrix}$$

verification – LxU

$$= \begin{bmatrix} 34.00 & 40.00 & 24.00 & 21.00 \\ 40.00 & 68.00 & 40.00 & 36.00 \\ 24.00 & 40.00 & 34.00 & 26.00 \\ 21.00 & 36.00 & 26.00 & 27.00 \end{bmatrix}$$

(b) cholesky decomposition $CA = L \cdot L^T$

```
def cholesky(A)
    n = len(A)
    L = [[0.0] *n for _ in range (n)]
    for i in range (n):
        for j in range (i+1):
            S = 0.0
            for k in range (j):
                S = S + L[i][k] * L[j][k]
            if i == j
                L[i][i] = (A[i][i] - S) ** 0.5
            else:
                L[i][j] = (A[i][j] - S) / L[j][j]
    return L
```

Date...../...../.....

Page.....

Cholesky factor $L = \begin{bmatrix} 5.83 & 0.00 & 0.00 & 0.00 \\ 6.85 & 4.57 & 0.00 & 0.00 \\ 4.11 & 2.57 & 3.23 & 0.00 \\ 3.60 & 2.46 & 1.49 & 2.38 \end{bmatrix}$

Verification $-L \times L^T = \begin{bmatrix} 34.00 & 40.00 & 24.00 & 21.00 \\ 40.00 & 68.00 & 40.00 & 36.00 \\ 24.00 & 40.00 & 34.00 & 26.00 \\ 21.00 & 36.00 & 26.00 & 27.00 \end{bmatrix}$

(c) QR Decomposition Via Gram-Schmidt

```
def dot(u,v):
    return sum(u[i]*v[i] for i in range(len(u)))

def norm(v):
    return dot(v,v) ** 0.5

def qr_gram_schmidt(A, m, n)
    Q=[[0.0]*n for _ in range(m)]
    R=[[0.0]*n for _ in range(n)]

    for j in range(n):
        v = [A[i][j] for i in range(m)]
        for i in range(j):
            q_i = [Q[r][i] for r in range(m)]
            a_j = [A[r][j] for r in range(m)]
            R[i][j]= dot(q_i, a_j)
            for r in range(m):
                v[r] = v[r] - R[i][j] * Q[r][i]
```

Date...../...../.....

Page.....

```
R[j][j] = norm(v)
for r in range(m):
    Q[r][j] = v[r] / R[j][j]
return Q, R

Q, R = qr_gram_schmidt(A, m, n)
```

d QR of 7x5 Random matrix: Results and observation.

matrix A (7x5):

2	4	9	9	3
5	6	5	2	8
4	8	2	2	1
7	5	8	3	1
9	9	5	8	6
5	2	1	6	4
4	7	3	6	1

Q matrix (7x5) - orthonormal columns.

0.13	0.29	0.79	0.44	0.07
0.34	0.12	0.09	-0.38	0.79
0.27	0.58	-0.29	-0.20	-0.18
0.47	0.38	0.34	-0.49	-0.48
0.61	-0.04	-0.22	0.18	0.06
0.34	-0.48	-0.21	0.50	0.11
0.27	0.49	-0.13	0.28	-0.27

Date...../...../.....

Page.....

R matrix (5×5) – Upper triangular

$$\begin{bmatrix} 14.69 & 15.24 & 11.49 & 12.45 & 9.18 \\ 0.00 & 6.53 & 2.25 & 2.32 & 0.30 \\ 0.00 & 0.00 & 8.46 & 4.08 & 0.90 \\ 0.00 & 0.00 & 0.00 & 7.54 & 0.95 \\ 0.00 & 0.00 & 0.00 & 0.00 & 6.46 \end{bmatrix}$$

Diagonal element of R:

[14.69, 6.53, 8.46, 7.54, 6.46]

~~Ex~~

$Q \times R = A$

$Q^T \times Q = I_5$ (columns orthonormal)